

Cambiar la imagen de predicción en YOLO

Introducción a la Inteligencia Artificial

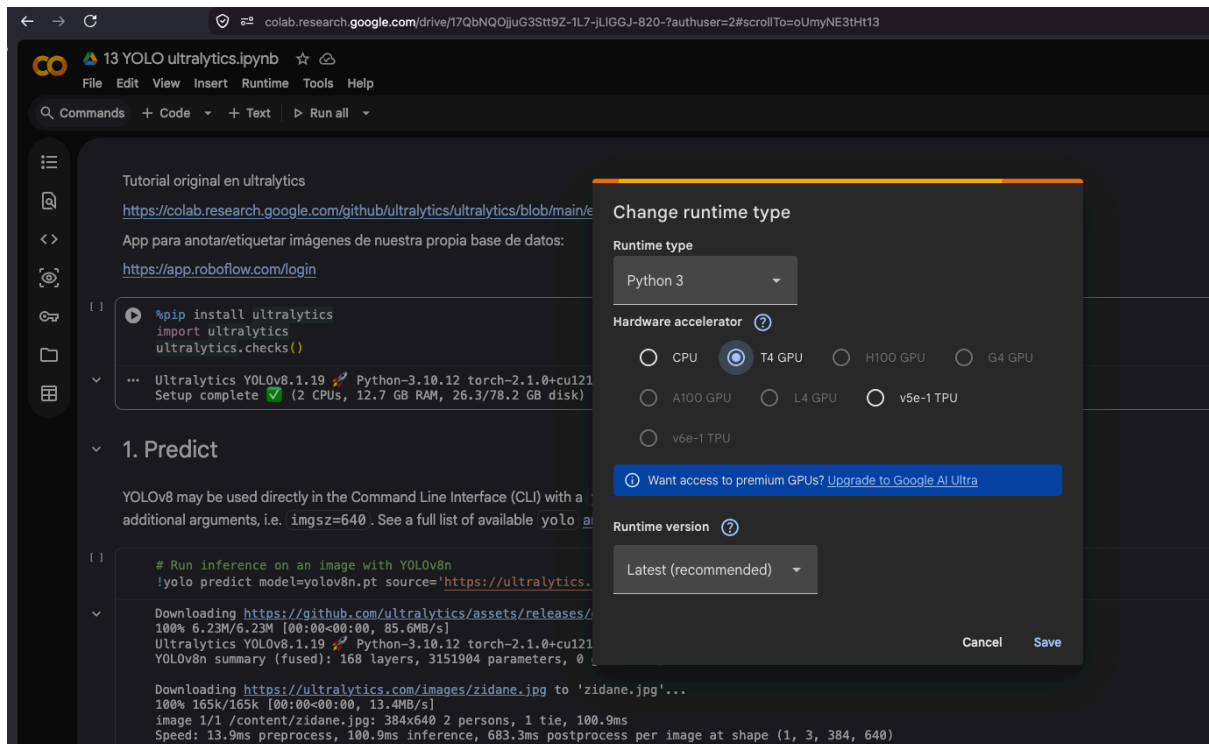
Zeus Sarmiento

11/09/2026

Enlace de Colab

- <https://colab.research.google.com/drive/17QbNQOjjuG3Stt9Z-1L7-jLIGGJ-820-?usp=sharing>

Capturas de ejecución resultados



```
... Ultralytics 8.4.150 Python-3.13.15 torch-2.11.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
Setup complete (2 CPUs, 12.7 GB RAM, 47.4/112.6 GB disk)
```

```
# Run inference on an image with YOLOv8n
!yolo predict model=yolov8n.pt source='https://ultralytics.com/images/zidane.jpg'

... Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolov8n.pt to 'yolov8n.pt': 100% 6.2MB 99.2MB/s 0.1s
Ultralytics 8.4.150 Python-3.13.15 torch-2.11.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
YOLOv8n summary (fused): 72 layers, 3,151,904 parameters, 0 gradients, 8.7 GFLOPs

Downloading https://ultralytics.com/images/zidane.jpg to 'zidane.jpg': 100% 49.2KB 6.7MB/s 0.0s
image 1/1 /content/zidane.jpg: 384x640 2 persons, 1 tie, 6.6ms
Speed: 292.9ms preprocess, 6.6ms inference, 41.3ms postprocess per image at shape (1, 3, 384, 640)
Results saved to /content/runs/detect/predict
```

```

[[161, 171, 188],
[160, 170, 187],
[160, 170, 187]],

...,

[[183, 182, 186],
[179, 178, 182],
[180, 179, 183],
...,
[121, 111, 117],
[113, 103, 109],
[115, 105, 111]],

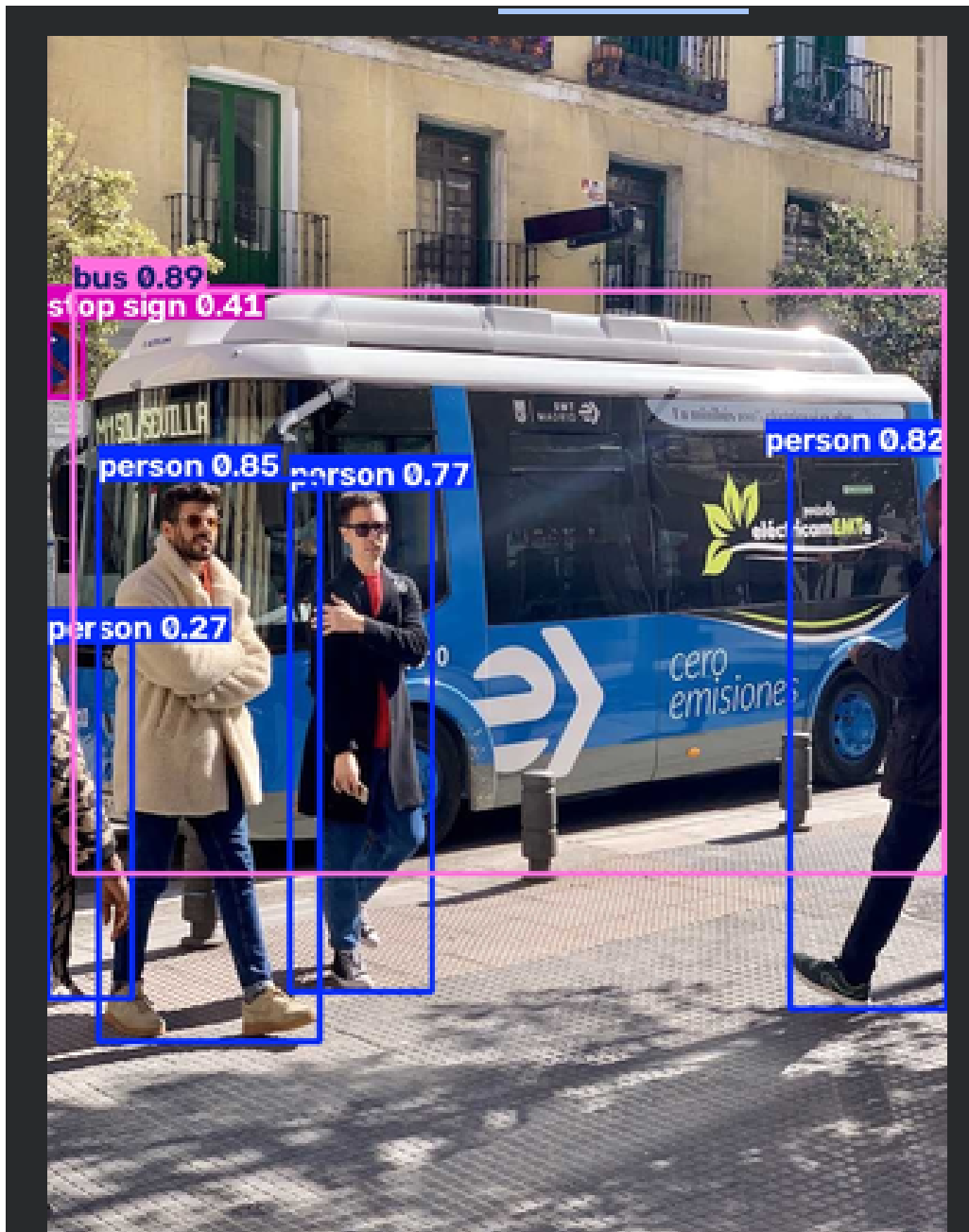
[[165, 164, 168],
[173, 172, 176],
[187, 186, 190],
...,
[102, 92, 98],
[101, 91, 97],
[103, 93, 99]],

[[123, 122, 126],
[145, 144, 148],
[176, 175, 179],
...,
[ 95, 85, 91],
[ 96, 86, 92],
[ 98, 88, 94]], dtype=uint8)
orig_shape: (1080, 810)
path: '/content/bus.jpg'
probs: None
save_dir: '/content/runs/detect/predict-2'
semantic_mask: None
speed: {'preprocess': 2.384892000009131, 'inference': 6.657583999867711, 'postprocess': 1.364101000035589}]

```

zidane.jpg





```
# Run inference on an image with YOLOv8n
!yolo predict model=yolov8n.pt source='https://ultralytics.com/images/mifoto.JPEG'

Ultralytics 8.4.150 Python-3.13.15 torch-2.11.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
YOLOv8n summary (fused): 72 layers, 3,151,904 parameters, 0 gradients, 8.7 GFLOPs

Found https://ultralytics.com/images/mifoto.JPEG locally at mifoto.JPEG
image 1/1 /content/mifoto.JPEG: 640x384 1 person, 1 bottle, 1 cup, 1 dining table, 1 tv, 1 cell phone, 10.6ms
Speed: 85.1ms preprocess, 10.6ms inference, 35.7ms postprocess per image at shape (1, 3, 640, 384)
Results saved to /content/runs/detect/predict-4
🔗 Learn more at https://docs.ultralytics.com/modes/predict

# Load YOLOv8n, train it on COCO128 for 3 epochs and predict an image with it
from ultralytics import YOLO

model = YOLO('yolov8n.pt') # load a pretrained YOLOv8n detection model
model.train(data='coco128.yaml', epochs=3) # train the model
model('https://ultralytics.com/images/mifoto.JPEG', save=True) # predict on an image

... Show hidden output
```



person 0.91

tv 0.40

bottle 0.60

lining table 0.45

cup 0.67

cell phone 0.49

Análisis de resultados

Corrí primero la notebook tal cual, con las fotos de Ultralytics, y después le pasé la misma foto mía por la celda del CLI y por la de model(...). La idea de la actividad, por lo que entendí, no era reentrenar YOLO ni cambiar el modelo, sino ver qué nombra cuando la imagen cambia y darse cuenta de que no “entiende” la foto: solo busca clases que ya trae de COCO y les pone caja.

En zidane.jpg salieron 2 person y 1 tie. Las dos personas se ven claro; la corbata sale con una confianza más baja (como 0.29), pero sí la marca. En bus.jpg marcó 1 bus, 1 stop sign y 4 person. Una de esas personas va recortada a la derecha y otra sale con poca confianza (0.27), entonces a veces etiqueta cosas que apenas se alcanzan a ver.

En la mía coincidió en lo pesado: person, bottle (el spray), cup (el portalápices), dining table (el escritorio), tv y cell phone. En el log de Python apareció además un knife, pero en la imagen no se ve una caja clara para eso.

De lo mío, tres cosas se me quedaron marcadas. El lapicero de la mesa se ve bien y no le puso etiqueta; me parece que o lo consideró muy chico, o no tiene una clase para “pluma” y por eso se lo saltó. La cartera sí la vio, pero la metió como cell phone con 0.49 de confianza: o sea, dudó y aun así la acomodó en lo más parecido que tenía. Y lo de atrás lo nombró tv, cuando en realidad es un monitor; COCO no trae “monitor” y lo más cercano acaba siendo tele.

Parece como que si el objeto está chico, si la confianza queda a medias o si la clase exacta no está en COCO, YOLO lo ignora o le pone otro nombre.

La predicción del CLI y la de model(...) sobre la misma foto sí coinciden en las clases principales. No fue que una viera un mundo y la otra otro. La diferencia fue ese knife extra en Python. Me queda que las dos celdas son la misma herramienta por caminos distintos; lo que cambia de verdad el resultado es la foto que le pases, no si lo corriste con !yolo o con model(...).

Evidencia de ejecución

